The Python Script (PythonScript) Agent

1 Introduction

The PythonScript agent is designed to allow for execution of Python scripts. Scripts will be run outside of the agent server process to protect the agent server process. Python scripts will have the ability to communicate to the agent server process via Adroit's OLE Automation component.

Future versions of the agent may allow for embedded running of python scripts.

2 Operation

Python must be installed on any agent server PC's requiring to run PythonScript agents.

The Adroit 11 installer will ensure that Python application as well as the pywin32 package is installed on the agent server computer. However, if Python has already been installed, the agent allows you to configure the path to python.exe.

On installing Adroit, you will have had the option of installing the Python environment, if you do not already have Python installed. Adroit will install Python to your C:\Program Files\Adroit Technologies\Python folder together with the pywin32 and adroit-py (Adroit OLE Python) packages. Python scripts are able to interact with the agent server using Adroit OLE Python package.

If you have Python already installed, you will need to

- install the PyWin32 package as follows:
  pip install pywin32
- install the Adroit OLE Python package as follows:
  pip install adroit-py

The Adroit OLE Python package (zip file) can also be download from [*to be decided*]. The Adroit OLE Python zip file contains:

- adroit_py-0.1.0-py3-none-any.whl - this file must be copied to your pc and installed as follows:

  pip install adroit_py-0.1.0-py3-none-any.whl

- myscript.py – a sample python script showing how to use the Adroit OLE package

The agent then allows you to specify the full path to the python script to be run. Python scripts should include the provided adroitole.py script for programmatic access to Adroit's OLE Automation component which will allow you to get (fetch) and set (poke) tag values.

The python script will run without a visible console window. Therefore …

- The use of print() statements in your scripts will never be visible
- **Do not use input() statements** in your python script otherwise the python.exe instance will remain in process space until manually terminated!!

The script can be configured to run synchronously or asynchronously. The script is always invoked on a separate thread to avoid holding up other agent server processes. In asynchronous mode the thread will terminate immediately after setting off the python process. In synchronous mode the thread will only terminate after the python process has terminated.

Once the thread is created to launch the script, a busy status bit will be set until the thread is terminated. Before terminating the thread will pulse the completed status bit to indicate completion.

The agent supports a list of trigger tags. Triggers tags may be individually enabled/disable. For each trigger tag the type of trigger may be configured viz any value change, rising edge (0->n) or falling edge (n->0).

Other than the ability to add trigger tags, this agent does not cater for scheduling of scripts

This agent does not support realtime catchup.

3 Scripting examples

3.1 Simple script

The following code shows the ability to execute a script with or without parameters

```
import sys
```

```
def HelloWorld():
    print("Hello World!")

def DoSomething(str):
    print(str)

# --- simply running a script file with no parameters
if __name__ == "__main__":
    if len(sys.argv) == 1:
        HelloWorld()
        sys.exit(0)

# --- running a script file with parameters DoSomething “Hello World”
    if len(sys.argv) == 3 and sys.argv[1] == " DoSomething ":
        DoSomething (sys.argv[2])
        sys.exit(0)

# --- if we get this far than the script was not called correctly, set agent’s error status to true
#     by exiting with a a non-zero code.  This will also show cause the agent’s info slot to show
#     “Process finished with error code: 1”
sys.exit(1)
```

3.2 Advanced script showing agent server interaction

The following code shows how to interact with Adroit

```
# Sample script demonstrating the use of AdroitOLE COM wrapper to interact with Adroit SCADA system.
# This script creates analog agents, sets alarms, scans tags, logs data, and performs various
operations.
# Revision: 1.0
# Date: 2026-01-22

from datetime import datetime, timedelta
from adroit import AdroitOLE
```

```
import random
import time

# instiantiate the AdroitOLE COM wrapper
from adroit import AdroitOLE
adroit:AdroitOLE = AdroitOLE();

def main():

    try:
        # Connect to a remote Agent Server named SERVER01:ADROIT
        #adroit.Connect("SERVER01","ADROIT")

        # Generate a random number and set it to MyAnalog2.value
        num = random.randint(1, 1000)

        # Set the random value to MyAnalog2.value
        adroit.SetTag("MyAnalog2.value", num)

        # Fetch back the value to verify
        result = adroit.Fetch("MyAnalog2", "value")
        print(f"\r\nSet random value {num} to MyAnalog2.value, fetched back: {result}")

        input("\r\nPress any key to continue...")

        # Fetch multiple tags MyAnalog1.value and MyAnalog2.value
        results = adroit.FetchTags(["MyAnalog1.value","MyAnalog2.value"])
        print("\r\nFetched Tags values for MyAnalog1.value and MyAnalog2.value:", results)

        input("\r\nPress any key to continue...")

        # Fetch slot info for agent MyAnalog1
        results = adroit.GetSlotInfo("MyAnalog1")
        print("\r\nGet Slot Info for agent MyAnalog1:", results)

        input("\r\nPress any key to continue...")

        print("\r\nSetting multiple values (50) to MyAnalog2.value...Please wait.")
        # Setting multiple values (50) to MyAnalog2.value
        for i in range(50):
            # Generate a random number and set it to MyAnalog2.value
            num = random.randint(1, 1000)

            # Set the value to MyAnalog2.value
            adroit.SetTag("MyAnalog2.value", num)

            time.sleep(0.3)  # slight delay to simulate time between setting values

        # Fetch changes for MyAnalog2.value between two dates (last 5 seconds)
        results = adroit.FetchChanges("MyAnalog2.value", (datetime.now() -
timedelta(seconds=5)).strftime("%Y-%m-%d %H:%M:%S"), datetime.now().strftime("%Y/%m/%d %H:%M:%S"))
        print("\r\nFetch Changes for MyAnalog2.value during last 5 seconds:", results)

        input("\r\nPress any key to continue...")

        # Fetch values for MyAnalog2.value between two dates (last 5 seconds), limit to 5 results
        results = adroit.FetchValues("MyAnalog2.value", (datetime.now() -
timedelta(seconds=5)).strftime("%Y-%m-%d %H:%M:%S"), datetime.now().strftime("%Y/%m/%d %H:%M:%S"), 5)
        print("\r\nFetch Values for MyAnalog2.value during last 5 seconds, limit to 5 results:",
results)

    except Exception as e:
        # Catch and print any errors
        print(f"Error: {e}")

    input("\r\nScript completed. Press Enter to exit...")

if __name__ == "__main__":
    main()
```

Please note when running your Adroit enabled scripts from the command-line (cmd.exe) for testing purposes, you must be running the command-line window as administrator (elevated mode) otherwise you will not be able to communicate to the Adroit Agent Server which always runs in elevated mode.

4 Agent Slots

As of Rev

4.1 Agent Specific Slots

The following slots are available in the scheduler agent:

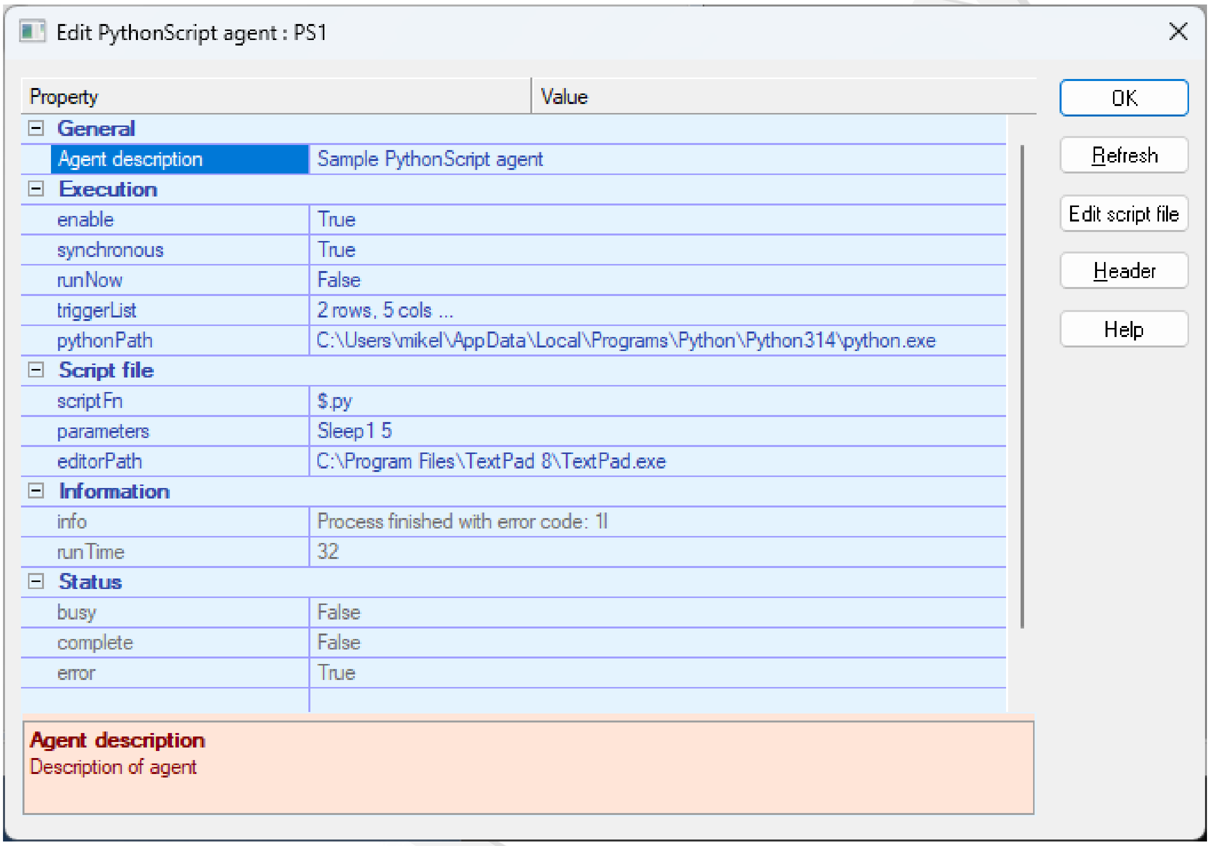
Slot name	Slot type	Slot description
autoReload	BOOLEAN	Monitor python script file timestamp and reload file if timestamp changes (for embedded mode only)
editorPath	STRING	Full file path to script code editor application – default `adrscriptedit.exe`
embedded	BOOLEAN	enable embedded mode – for future use only
enable	BOOLEAN	Enable agent
Info	STRING	Feedback information. If a script error occurs `sys.exit(`*nnn*`)` calls will set this slot value to show "Process finished with error code: *nnn*"
parameters	STRING	Optional parameters to be passed to script
pythonPath	STRING	Full path to Python.exe. Will default to `C:\Program Files\Adroit Technologies\Python\Python.exe` if it exists.
reloadNow	BOOLEAN	Reload script – only valid for embedded mode
runNow	BOOLEAN	Execute script now
runTime	BOOLEAN	Time taken for script to complete execution in ms
scriptFn	STRING	Filename of script to be launched. If scriptFn does not contain a full path, it will be searched for in the Adroit project data folder
synchronous	BOOLEAN	Run script synchronously – default 0
triggerList	LIST	List of trigger tags

4.2 Status slots

Status name	Status description
busy	Script has been launched and is running. In synchronous mode, bit will be turned off prior to setting complete status. In asynchronous mode, bit will be turned off when script has completed.
complete	Script has been launched (in synchronous mode) or completed (in asynchronous mode)
error	An error has occurred

5 Configuration

The configuration dialog will allow you to select and edit your python script file. By default, Adroit will attempt to open your code using adrscriptedit.exe (Scintilla host). Otherwise you can opt to use your own favourite script editor by specifying the full path to the editor executable.



The PythonScript agent dialog